# SQL Assistant – Technical Document

Oct 22, 2026 · @Anitha

## Overview

SQL Assistant turns a plain-English request into one SQLite query against the Northwind sample database, then shows the query and its result without ever changing the database. Version 1 is built and tested locally. It has not yet been tested against the live Groq API and is not deployed.

The page has three parts:

- **Scenario**: background for the request, for example "I'm reviewing late deliveries for the sales team". Optional.
- **Question**: the specific ask, for example "Show orders shipped after their required date". Required.
- **Query**: the generated SQL (read-only text box with a Copy button), a one-line explanation, and the result.

What the result shows depends on the query:

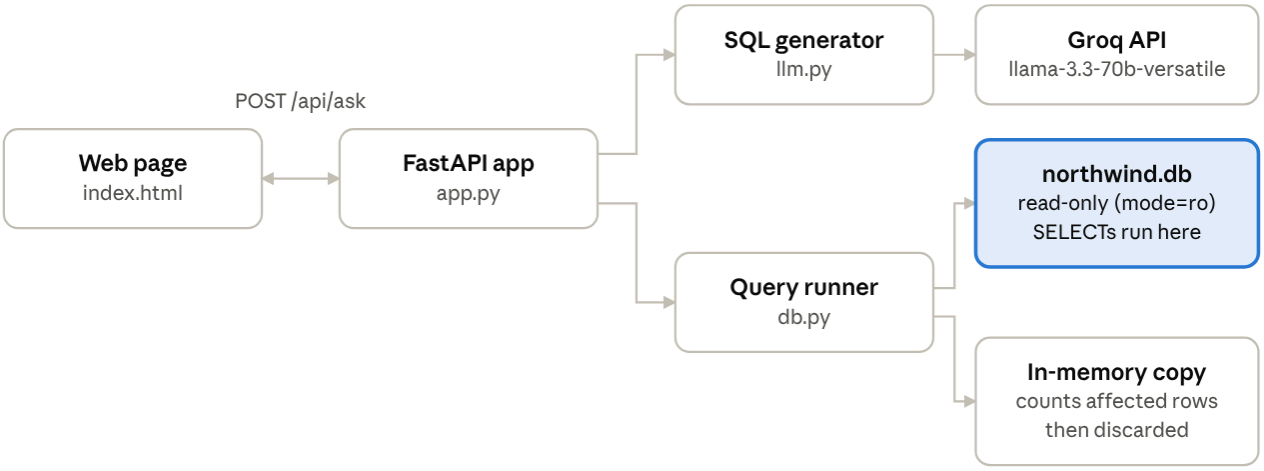
| Query type | What the app shows | Database changed? |
|---|---|---|
| SELECT, WITH, VALUES | The query and up to 200 result rows | No |
| INSERT, UPDATE, DELETE, REPLACE | The query and "This query would affect N records. It was not applied." | No |
| Anything else (DROP, CREATE, ALTER, PRAGMA, ATTACH…) | The query and "… statements are shown but not executed." | No |

Out of scope for version 1, as agreed: fixing a query the user pastes in, and dialects other than SQLite.

## Architecture and tech stack

The app is a small Python FastAPI server with one HTML page, Groq as the AI model, and Northwind as a bundled SQLite file.

**Every query reaches Northwind through a read-only path**




architecture · 7 parts

The page talks only to the FastAPI app. The app asks Groq for SQL, then hands the SQL to the query runner, which reads the real file read-only or does a dry run on a throwaway copy.

| Part | Choice | Why |
|---|---|---|
| Backend | Python, FastAPI | Runs as a Vercel serverless function; Streamlit needs a long-running server, which Vercel doesn't provide |
| Frontend | One HTML page with plain JavaScript and CSS, no framework | Small and fast; follows light or dark system theme; works at phone width |
| AI model | Groq, `llama-3.3-70b-versatile` | Chosen by Anitha; model can be changed with `GROQ_MODEL` |
| Database | Northwind for SQLite, 24.7 MB file shipped with the app | Free, needs no database server, can be opened read-only |
| SQL dialect | SQLite | Matches the database |

| File | What it does |
|---|---|
| `app.py` | FastAPI app: serves the page and the `/api/ask` endpoint |
| `llm.py` | Builds the prompt and calls Groq |

| File | What it does |
|---|---|
| `db.py` | Reads the schema, runs queries read-only, does the dry run |
| `templates/index.html` | The page: Scenario, Question, Query and results |
| `northwind.db` | The sample database |
| `requirements.txt` | `fastapi`, `groq` |
| `README.md` | Short setup guide |

## Request flow

Every click on **Generate query** makes one round trip: the page calls the server, the server asks Groq for SQL, then runs that SQL under the safety rules.

1. The page sends `POST /api/ask` with the scenario and question as JSON.
2. `app.py` checks the input (question 1 to 2,000 characters, scenario up to 2,000) and loads the database schema. The schema is read once per server instance and cached.
3. `llm.py` sends Groq a system prompt plus the full schema (every table and view with its columns), then the user's scenario and question. It uses `temperature=0` and JSON response mode.
4. The prompt tells the model to write exactly one SQLite statement, to use only names in the schema, and to return an empty `sql` with a reason if the question can't be answered from Northwind. It must not guess.
5. If the model returns SQL, `db.execute()` classifies it by its first keyword and runs it on the matching path (see the next section).
6. The server returns the SQL, the explanation and the result. The page fills the Query box and draws a table or a notice.

## Read-only safety design

The file `northwind.db` can't be changed by the app. The guarantee doesn't depend on the AI behaving: SQLite itself refuses writes on every connection to the real file. Several layers enforce it.

| Layer | Where | What it does |
|---|---|---|
| Read-only file mode | `db._open_readonly()` | Opens the file with `file:northwind.db?mode=ro`. SQLite rejects any write at the file level. |
| `PRAGMA query_only = ON` | `db._open_readonly()` | A second, connection-level block on writes. |
| Authorizer | `db._authorizer` | Denies `ATTACH`, `DETACH` and `PRAGMA` inside user queries, on both the read connection and the scratch copy. This stops a query from attaching or creating another database file. |
| In-memory dry run | `db._dry_run_write()` | INSERT/UPDATE/DELETE/REPLACE run on a throwaway `:memory:` copy made with SQLite's backup API. The app reads `total_changes`, rolls back and discards the copy. |
| One statement only | Python `sqlite3` | `execute()` refuses more than one statement ("You can only execute one statement at a time"), so `SELECT 1; DELETE ...` fails. |
| Time limit | `db._add_timeout()` | A progress handler stops any query after 10 seconds. |
| Row limit | `db._run_read()` | At most 200 rows are returned; the page says when there were more. |

How statements are routed:

- **Read keywords** (SELECT, WITH, VALUES) run on the read-only connection. A `WITH ... DELETE` statement fails there with a read-only error, and is then sent to the dry run instead.
- **Write keywords** (INSERT, UPDATE, DELETE, REPLACE) go straight to the dry run.
- **Everything else** is shown but not executed.

The dry-run count is SQLite's `total_changes`. It counts rows changed by the statement, including rows changed by triggers or cascades, if the schema defines any.

The schema reader ( `get_schema` ) is the one place that uses `PRAGMA table_info`. It runs only the app's own fixed queries, on a read-only connection without the authorizer.

# API reference

The app has two endpoints: one serves the page and one does the work.

| Method | Path | Purpose |
|---|---|---|
| GET | `/` | Returns `templates/index.html` |
| POST | `/api/ask` | Generates SQL and runs it safely |

**Request body for `POST /api/ask`:**

```
{"scenario": "I'm reviewing late deliveries", "question": "Show orders shipped after their required date"}
```

`question` is required (1 to 2,000 characters). `scenario` is optional (up to 2,000 characters). Invalid input returns HTTP 422 from FastAPI.

**Response body:** always `sql`, `explanation` and `result`. `result` is `null` when the model returned no SQL, for example when the question can't be answered from Northwind. Otherwise it takes one of these shapes:

| `result.kind` | Other fields | Meaning |
|---|---|---|
| `read` | `columns`, `rows` (max 200), `truncated` | A SELECT ran on the read-only connection |
| `write` | `affected_rows` | Dry-run count; nothing was saved |
| `not_run` | `message` | Statement type the app doesn't execute |
| `error` | `error` | SQL error, timeout, missing API key or a failed Groq call |

Example of a write result, from a test with `DELETE FROM Customers WHERE Country='Mexico'`:

```
{"sql": "DELETE FROM Customers WHERE Country='Mexico'", "explanation": "Deletes Mexican customers.", "result": {"kind": "write", "affected_rows": 5}}
```

## Configuration, setup and deployment

The app needs one secret, a Groq API key. It is read from an environment variable and is never stored in the code.

| Variable | Required | Default | Purpose |
|---|---|---|---|
| `GROQ_API_KEY` | Yes | none | Key from console.groq.com. Without it, every request returns "GROQ_API_KEY is not set". |
| `GROQ_MODEL` | No | `llama-3.3-70b-versatile` | Groq model ID. Listed as a production model in Groq's docs on 22 Oct 2026. |

Fixed settings in `db.py`: `MAX_ROWS = 200`, `TIMEOUT_SECONDS = 10`.

**Run locally** (Python 3; tested on 3.13):

```
pip install -r requirements.txt uvicorn
export GROQ_API_KEY=your-key-here
uvicorn app:app --reload
```

Then open http://127.0.0.1:8000. On Windows PowerShell, set the key with `$env:GROQ_API_KEY="your-key-here"`.

**Deploy to Vercel** (planned steps, not yet carried out):

1. Put the project folder in a Git repo and import it in Vercel, or run `vercel` in the folder. The FastAPI `app` object is in `app.py`, which Vercel's Python runtime is expected to detect.
2. In the Vercel project's Settings, under Environment Variables, add `GROQ_API_KEY`.
3. Deploy, then run the checks in the next section against the live URL.

`northwind.db` (24.7 MB) and `templates/index.html` must ship with the function, because the app reads both from its own folder at runtime. Whether Vercel bundles them without extra config is one of the things the first deploy will confirm.

`requirements.txt` lists `fastapi` and `groq` without pinned versions. Testing used fastapi 0.159.1 and groq 1.7.0.

## Testing, open items and limitations

The safety rules and the page passed local tests. The Groq step has only been tested with a stand-in, because no API key was available. The database file's checksum was the same before and after the whole test run.

| Test statement | Result |
|---|---|
| `SELECT ... FROM Customers WHERE Country='Germany'` | Rows returned |
| `UPDATE Products SET UnitPrice = UnitPrice*1.1 WHERE CategoryID=1` | Would affect 12 rows; not applied |
| `DELETE FROM [Order Details] WHERE Quantity > 100` | Would affect 13 rows; not applied |
| `INSERT INTO Shippers ...` | Would affect 1 row; not applied |
| `WITH x AS (...) DELETE FROM Shippers ...` | Routed to the dry run; would affect 1 row |
| `DROP TABLE Shippers` | Shown, not executed |
| `ATTACH DATABASE '/tmp/evil.db' AS e` | Shown, not executed; no file created |
| `SELECT 1; DELETE FROM Shippers` | Refused: one statement at a time |
| `SELECT * FROM pragma_table_info('Shippers')` | Refused by the authorizer |
| `SELECT * FROM NoSuchTable` | Error message shown |
| `SELECT * FROM [Order Details]` | First 200 rows, marked as truncated |
| Cross join of `[Order Details]` with itself | Stopped after 10 seconds |
| Missing `GROQ_API_KEY` | Clear error message, no crash |

The page was checked in a headless browser at desktop width (1100 px) and phone width (390 px), with the AI step replaced by a fixed answer.

**Still untested:**

- [ ] Real Groq responses: SQL quality, the empty-`sql` reply for questions outside Northwind, and response time. Needs a Groq key.
- [ ] Deployment on Vercel, including whether `northwind.db` and the page template are bundled.

**Known limitations:**

- **This is the extended Northwind dataset** (jpwhite3/northwind-SQLite3). Order dates run from 10 Jul 2012 to 28 Oct 2023, and `[Order Details]` has 609,283 rows. Some built-in views with fixed 1997 dates, such as `[Product Sales for 1997]`, return 0 rows.

- **No login or rate limit.** Once deployed, anyone with the URL can send questions, and each one uses your Groq quota.
- **Schema in every prompt.** Each request sends the full table and view list to Groq. This keeps the model to real names, but it adds tokens to every call.
- **Only the first keyword decides the path.** Unusual statements are shown but not run, rather than guessed at.
- **SQLite dialect only.** Queries may need changes to run on SQL Server or MySQL.